

DEVOPS

ASSIGNMENT -2

INSTRUCTIONS

Mandatory Submission Links

Students must include the following links on the **first page** of their documentation report.

Sl. No.	Submission Item	Description
1	GitHub Repository Link	Link to the complete source code repository.
2	Jenkins Build URL (<i>if accessible</i>)	URL of the successful Jenkins job or pipeline. If the Jenkins server is running locally, include screenshots of the Jenkins Dashboard, Job Configuration, Console Output, and Successful Build instead of the URL.
3	Docker Hub Repository Link (<i>Optional</i>)	If the Docker image has been pushed to Docker Hub, provide the repository link.
4	Application URL	URL of the deployed application (if deployed on a cloud platform). If running locally, provide the local URL (e.g., <code>http://localhost:<port></code>) along with a browser screenshot.
5	Grafana Dashboard Screenshot	Include screenshots of the dashboard showing the monitored metrics.
6	Nagios Monitoring Screenshot	Screenshot showing the monitored host and services in the UP/OK state.
7	Graphite Metrics Screenshot	Screenshot showing application or system metrics received in Graphite.

Files to be Submitted

Students must upload the following:

- Complete Project Folder (ZIP)
- Documentation Report (PDF or Microsoft Word)
- GitHub Repository Link
- Jenkins Automation Screenshots
- Docker Build and Running Container Screenshots
- Kubernetes Deployment Screenshots
- Nagios Monitoring Screenshots
- Graphite Metrics Screenshots
- Grafana Dashboard Screenshots
- Application Output Screenshots
- Step-by-Step Implementation Screenshots

Naming Convention

Use the following naming format for submissions:

- **Project ZIP:** RegisterNumber_Name_DevOps_Project.zip
- **Documentation:** RegisterNumber_Name_DevOps_Report.pdf
- **GitHub Repository:** RegisterNumber-DevOps-Project

Final Submission Checklist

Before submitting, ensure that:

- ☐ GitHub repository is accessible.
- ☐ All source code has been pushed to GitHub.
- ☐ Jenkins build completed successfully.
- ☐ Maven build completed successfully.
- ☐ Docker image was created successfully.
- ☐ Docker container is running.
- ☐ Kubernetes Pods and Services are in the **Running** state.
- ☐ Application is accessible in the browser.

- ☐ Nagios displays the Host as **UP** and Services as **OK**.
- ☐ Graphite is receiving metrics.
- ☐ Grafana dashboard displays the required metrics.
- ☐ Documentation contains screenshots of every implementation step.
- ☐ All required links have been included.

CHOOSE ANY ONE:

Use Case 1: Corporate Company Website Deployment

ABC Technologies has designed a new corporate website consisting of HTML, CSS, JavaScript, and image files. The website contains multiple pages, including Home, About Us, Services, Careers, Contact Us, and Gallery. The management wants the website to be automatically deployed whenever developers update the website content.

The organization has decided to implement a DevOps workflow that enables collaborative development, automated deployment, containerized hosting, orchestration, and continuous monitoring. The website should remain available even after updates, and administrators should be able to monitor the website's health and performance through monitoring dashboards.

Requirements

- Multiple developers should be able to collaborate on the website.
- Every website update should trigger an automated deployment process.
- The website should be hosted inside a Docker container.
- The container should be deployed in Kubernetes.
- The website should be accessible through a web browser.
- Website availability should be monitored continuously.
- Infrastructure and application metrics should be collected.
- Administrators should be able to visualize the website's health through dashboards.

Expected Final Output

- Source code stored in a Git repository.

- Automated build executed successfully.
- Docker container running.
- Kubernetes Pods and Services running.
- Website accessible through browser.
- Nagios displays **Host UP** and **HTTP Service OK**.
- Graphite receives monitoring metrics.
- Grafana dashboard displays CPU, Memory, Network Usage, HTTP Availability, and Uptime.

Use Case 2: College Event Website

A college is organizing its annual technical symposium and has developed a website to publish event schedules, registration information, speaker details, and announcements. Since information changes frequently, the website should automatically reflect updates without manual deployment.

The institution wants a complete DevOps implementation to automate website deployment while ensuring continuous monitoring and high availability.

Requirements

- Maintain version history for all website changes.
- Automate deployment whenever new updates are committed.
- Host the website using Docker.
- Deploy the website using Kubernetes.
- Ensure the website is available to users at all times.
- Monitor website availability.
- Collect server resource metrics.
- Display monitoring dashboards.

Expected Final Output

- Website deployed successfully.
- Running Docker container.
- Kubernetes Deployment and NodePort Service created.

- Website accessible through browser.
- Nagios reports website availability.
- Graphite stores performance metrics.
- Grafana dashboard displays CPU, Memory, Disk Usage, Network Traffic, and Uptime.

Use Case 3: Online Portfolio Website

A web design company maintains portfolio websites for multiple clients. Designers frequently update project images, testimonials, and service information. The company wants every update to be deployed automatically with minimal downtime.

To improve operational efficiency, the company has decided to implement a complete DevOps pipeline for website deployment and monitoring.

Requirements

- Store website files in a centralized version control repository.
- Automate deployment after every approved code update.
- Host the website inside a Docker container.
- Deploy using Kubernetes.
- Continuously monitor website availability.
- Collect infrastructure metrics.
- Display resource utilization dashboards.

Expected Final Output

- Website deployed successfully.
- Running Kubernetes Pods.
- Website accessible using browser.
- Nagios monitoring website availability.
- Graphite collecting metrics.
- Grafana displaying dashboards.

Use Case 4: Company Intranet Portal

A multinational company maintains an internal employee portal that provides company news, HR policies, holiday calendars, and announcements. The portal is updated regularly by the HR department.

The IT department wants to automate deployment and monitoring to reduce manual effort and ensure that employees can always access the portal.

Requirements

- Maintain all website files in version control.
- Automate deployment after updates.
- Deploy using Docker and Kubernetes.
- Monitor website uptime.
- Collect infrastructure metrics.
- Display dashboards for administrators.

Expected Final Output

- Successful deployment.
- Website available through browser.
- Kubernetes Deployment running.
- Nagios monitoring active.
- Graphite collecting metrics.
- Grafana dashboard showing system health.

Use Case 5: Tourism Information Website

A state tourism department has created a website showcasing tourist attractions, hotels, travel guides, maps, and booking links. During holiday seasons, thousands of visitors are expected to access the website simultaneously.

The department requires a DevOps solution that enables automated deployment, scalable hosting, and continuous monitoring to ensure uninterrupted service availability.

Requirements

- Collaborative source code management.

- Automated deployment workflow.
- Docker-based hosting.
- Kubernetes deployment.
- Continuous website monitoring.
- Infrastructure metric collection.
- Dashboard visualization.

Expected Final Output

- Website accessible through browser.
- Docker container running successfully.
- Kubernetes Pods in Running state.
- NodePort Service configured.
- Nagios reporting website availability.
- Graphite storing performance metrics.
- Grafana displaying CPU, Memory, Network, and Uptime dashboards.